



پردیس دانشکده‌های فنی

دانشگاه تهران



رباتیک پیشرفته - نیمسال دوم ۱۴۰۳-۴

فاز سوم پروژه - پیاده‌سازی الگوریتم Particle Filter

نگارش:

علی خیری 810603071

معین رضایی 810602081

محمد رضا توکلی 159403003

استاد درس:

دکتر نصیری - دکتر مرادی

تابستان ۱۴۰۴

فهرست

۴.....	ساختن نقشه محیط
۶.....	راه اندازی ربات و نقشه
۱۰.....	Particle Initialization – شروع ذرات
۱۱.....	Motion Model – مدل حرکتی
۱۵.....	Particles Sensor Reading – مدل سنسوری ذرات
۱۶.....	Weight Update – بروزرسانی وزنها
۱۸.....	Resampling – باز نمونه گیری
۱۸.....	Pose Estimation – تخمین موقعیت
۱۹.....	بهینه سازی پارامتری نویزی
۱۹.....	قراردادن بخشهای مختلف – الگوریتم نهایی
۲۳.....	نتایج
۲۶.....	تشخیص kidnapping ربات و بازیابی الگوریتم:
۲۶.....	الگوریتم ۱:
۲۹.....	الگوریتم ۲:
۳۳.....	پیوست – ورژنهای مختلف الگوریتم

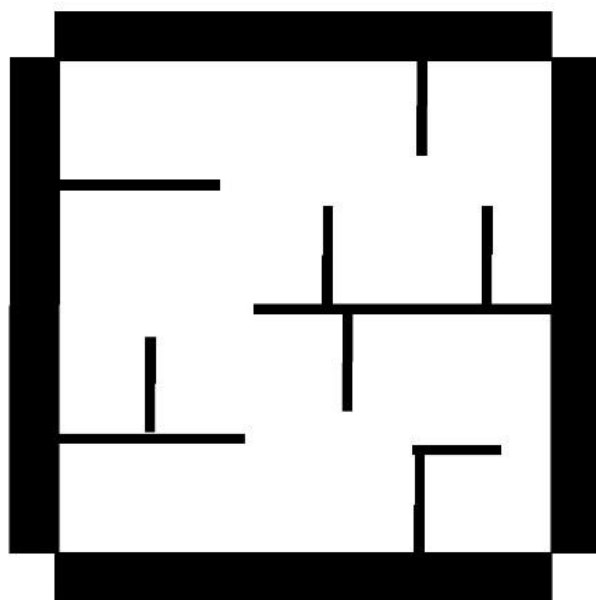
فهرست شکل ها

- شکل ۱ تصویر crop شده از نقشه maze به فرمت pgm ۵
- شکل ۲ تصویر ربات در محیط Gazebo ۷
- شکل ۳- محیط rviz ۷
- شکل ۴ اضافه کردن نقشه ۸
- شکل ۵ اضافه کردن لیزر ۸
- شکل ۶ اضافه کردن مدل ربات ۹
- شکل ۷ اضافه کردن پارتیکل ها ۹
- شکل ۸ - ذرات اولیه در محیط rviz ۱۱
- شکل ۹ - بررسی مدل حرکتی ذرهها در حرکت مستقیم - حالت اولیه ۱۳
- شکل ۱۰ - بررسی مدل حرکتی ذرهها در حرکت مستقیم - حالت ثانویه ۱۳
- شکل ۱۱ - بررسی مدل حرکتی ذرهها در حرکت چرخشی ۱۴
- شکل ۱۲ - بررسی مدل سنسوری ذرهها - ۱ ۱۵
- شکل ۱۳ - بررسی مدل سنسوری ذرهها - ۲ ۱۶
- شکل ۱۴ - آپدیت وزنها ۱۷
- شکل ۱۵ - محیط ترمینال پس از موقعیتیابی کامل ۲۰
- شکل ۱۶ - ذره با بیشترین وزن به رنگ طلایی ۲۱
- شکل ۱۷ - نمودارهای خطا در تست ۱۰ ۲۵
- شکل ۱۸- در ابتدا ربات شروع به حرکت می کند تا موقعیت یابی کند(فاز اول) ۲۶
- شکل ۱۹- پارتیکل ها آرام آرام همگرا می شوند و خطا کاهش می یابد(فاز دوم) ۲۷
- شکل ۲۰ ربات به صورت ناگهانی جا به جا می شود و kidnap تشخیص داده می شود.(فاز سوم) ۲۷
- شکل ۲۱- پارتیکل ها کانورج میکنند تا دوباره موقعیت یابی از سر گرفته شود(فاز چهارم) ۲۸
- شکل ۲۲ حالت چالشی برای تفکیک از ربوده شدن ۳۰
- شکل ۲۳ مکانیزم توقف بروزرسانی ۳۱
- شکل ۲۴- داده های تغییرات میانگین وزن ۲۰ درصد برتر در حالت ربوده شدن ۳۱

ساختن نقشه محیط

همان طور که مشخص است ابتدا که فایل `start_world.launch` را ران می‌کنیم. مشاهده می‌کنیم که یک محیط `maze` و ربات در این محیط قرار دارد. محیط سه بعدی برای استفاده `particle filter` به تنهایی ناکارآمد است به همین دلیل نیاز بود که یک نقشه دوبعدی از محیط تهیه کنیم. برای تهیه نقشه دوبعدی باید ابعاد ۱۴ بلوک تشکیل دهنده `maze` را به همراه مرکز این بلوک‌ها نسبت به محور مختصات `global` داشته باشیم. تمامی این ابعاد را از فایل `world` درون نقشه استخراج کردیم و آن را در یک لیست ریختیم. که به صورت زیر نمایش داده شده است. ذکر این نکته هم قابل توجه است که مرکز اولین بلوک را به عنوان مرکز `global` گرفتیم تا محور اصلی ربات در `gazebo` منطبق شود. و کد با نام `map_generator.ipynb` ذخیره شده است که توضیح آن به این صورت است.

این کدی که نوشته‌ایم برای تولید یک نقشه‌ی دو بعدی از یک ماز (`maze`) طراحی شده است که بتوان در ادامه از آن به عنوان محیطی برای پیاده‌سازی الگوریتم فیلتر ذره‌ای (`Particle Filter`) استفاده کرد. هدف اصلی از تولید این نقشه، داشتن تصویری دقیق و باینری از دیوارها و مسیرهای آزاد در محیط است، به طوری که هم قابل استفاده در `ROS` باشد و هم بتوان آن را در الگوریتم مکانیابی به کار برد. در ابتدای کد، دو کلاس اصلی تعریف شده‌اند: کلاس `Wall` و کلاس `Maze`. هر شیء از کلاس `Wall` نمایانگر یک دیوار مجزا در ماز است و اطلاعاتی مثل مختصات مرکز دیوار، زاویه چرخش (`yaw`)، طول و عرض دیوار را در خود نگه می‌دارد. متد `get_corners` در این کلاس برای محاسبه‌ی مختصات چهار گوشه‌ی هر دیوار طراحی شده است که با استفاده از ماتریس چرخش، موقعیت گوشه‌ها را با توجه به زاویه‌ی دیوار نسبت به محور مختصات محاسبه می‌کند. کلاس `Maze` کل ساختار ماز را تعریف می‌کند. این کلاس لیستی از دیوارها را به صورت دستی مقداردهی کرده است تا نمای کلی ماز را بسازد. هر دیوار با موقعیت دقیق و زاویه مشخص در محیط قرار داده شده است. پارامترهایی مانند اندازه‌ی کلی نقشه (`map_size_m`) و وضوح نقشه (`resolution`) مشخص می‌کنند که نقشه خروجی با چه ابعادی و دقتی ترسیم شود. همچنین مبدأ نقشه (`origin`) مشخص می‌کند که مختصات دنیای واقعی چگونه به مختصات تصویر تبدیل شوند. تابع `generate_map` مسئول تولید سه خروجی اصلی است:



شکل ۱ تصویر crop شده از نقشه maze به فرمت pgm

فایل تصویری PGM از ماز، فایل YAML شامل اطلاعات از جمله مختصات مرکز و رزولوشن نقشه برای استفاده در ROS، و فایل باینری npy که نسخه‌ی عددی و ساده‌شده‌ی نقشه برای استفاده در الگوریتم فیلتر ذرات محسوب می‌شود. در این تابع ابتدا یک تصویر سفید با استفاده از کتابخانه‌ی PIL ایجاد می‌شود که هر پیکسل آن نشان‌دهنده یک نقطه در فضای واقعی است. سپس با استفاده از تابع `world_to_pixel`، مختصات هر گوشه‌ی دیوارها از فضای واقعی به فضای پیکسلی تبدیل می‌شوند. این کار باعث می‌شود که بتوان موقعیت دقیق دیوارها را روی تصویر رسم کرد. پس از رسم همه‌ی دیوارها، تصویر به صورت یک فایل PGM ذخیره می‌شود که با فرمت نقشه‌های قابل استفاده در ROS سازگار است. در مرحله‌ی بعد، فایل YAML تولید می‌شود که اطلاعات مورد نیاز ROS را در اختیار قرار می‌دهد؛ مثل مسیر فایل تصویر، وضوح، مبدأ، و آستانه‌هایی برای تشخیص ناحیه‌های اشغال‌شده یا آزاد. در نهایت تصویر به آرایه‌ای از اعداد تبدیل می‌شود و با آستانه‌گذاری ساده (بیشتر از ۱۲۸ یعنی سفید و کمتر مساوی ۱۲۸ یعنی دیوار) به نقشه‌ای باینری تبدیل می‌شود که تنها شامل ۰ و ۱ است. این نقشه برای الگوریتم فیلتر ذره‌ای که نیاز به نقشه‌ای ساده و قابل فهم برای پردازنده دارند، بسیار مفید است. در مجموع این کد با دقت بالا مازی را که به صورت دستی تعریف شده، به سه فرمت مختلف برای استفاده در پروژه شبیه‌سازی تولید می‌کند. جدول زیر اطلاعات موجود در فایل Yaml است.

جدول ۱ پارامتر های موجود در فایل yaml و توضیح هر پارامتر

پارامتر	مقدار	تعریف
Image	Map.pgm	نام فایل تصویری نقشه که با فرمت PGM ذخیره می‌شود.
Resolution	0.001	وضوح نقشه به واحد متر بر پیکسل؛ هر پیکسل معادل ۱ میلی‌متر در دنیای واقعی است.
Origin	[-1.25, -1.25, 0]	مبدأ نقشه در مختصات دنیای واقعی به صورت [x, y, yaw]
Negate	0	اگر برابر ۱ باشد، رنگ‌های سیاه و سفید در تصویر معکوس می‌شوند؛ در اینجا نیازی به معکوس‌سازی نیست.
Occupied_thresh	0.65	آستانه‌ای که بالاتر از آن، یک پیکسل به عنوان ناحیه اشغال شده (دیوار) شناخته می‌شود.
Free_thresh	0.196	آستانه‌ای که پایین‌تر از آن، یک پیکسل به عنوان ناحیه آزاد شناخته می‌شود.

راه اندازی ربات و نقشه

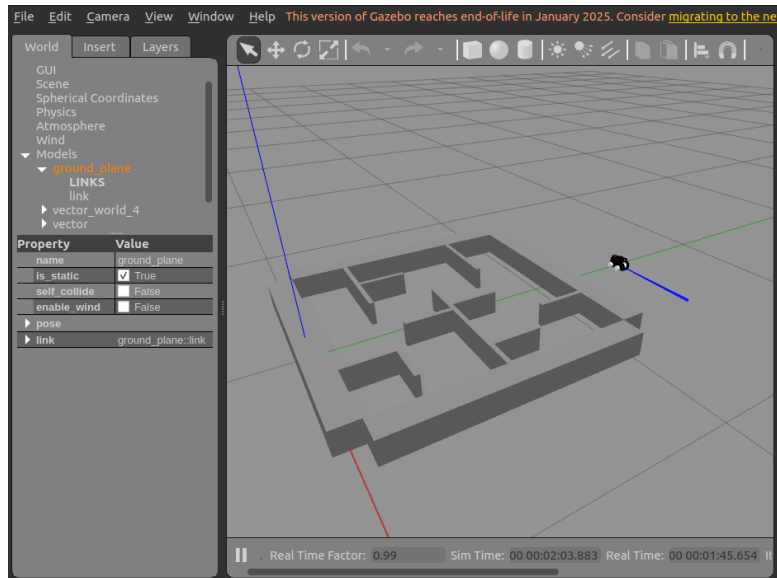
به منظور راه اندازی ربات به همراه محیط maze ابتدا فایل start_world.launch موجود در فایل anki_description که پیشتر آن را clone کردیم از طریق کامند زیر در ترمینال ران می‌کنیم. فقط پیش از این باید حتماً setup.bash را source کنیم.

```
>> cd catkin_ws
```

```
>> source devel/setup.bash
```

```
>> roslaunch anki_description start_world.launch
```

به این شکل ربات در محیط Gazebo ران خواهد شد.



شکل ۲ تصویر ربات در محیط Gazebo

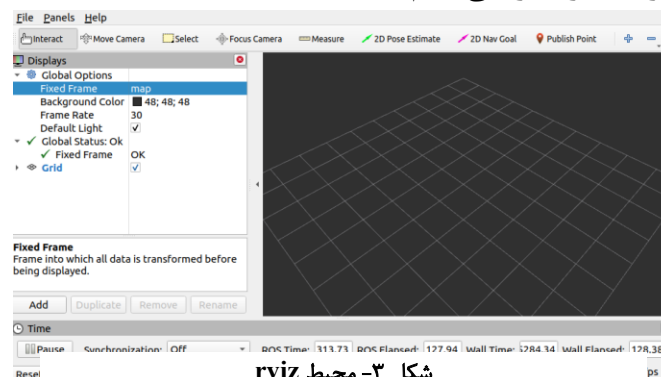
حال نقشه محیط را در rviz بالا می‌آوریم. برای این کار با استفاده از نصب پکیج map_server فایل yml را اجرا می‌کنیم.

```
>> rosruncatkin_ws/src/anki_description/maps/map.yml
```

به منظور اینکه نقطه دستگاه مختصات نقشه را در محیط rviz منطبق بر نقطه (0, 0, 0) قرار دهیم از دستور زیر استفاده می‌کنیم. که ۳ صفر ابتدایی مربوط به محورهای مختصاتی xyz و ۳ صفر بعدی مربوط به جهت‌گیری زاویه‌ای ربات در فضا یا همان roll, pitch, yaw است.

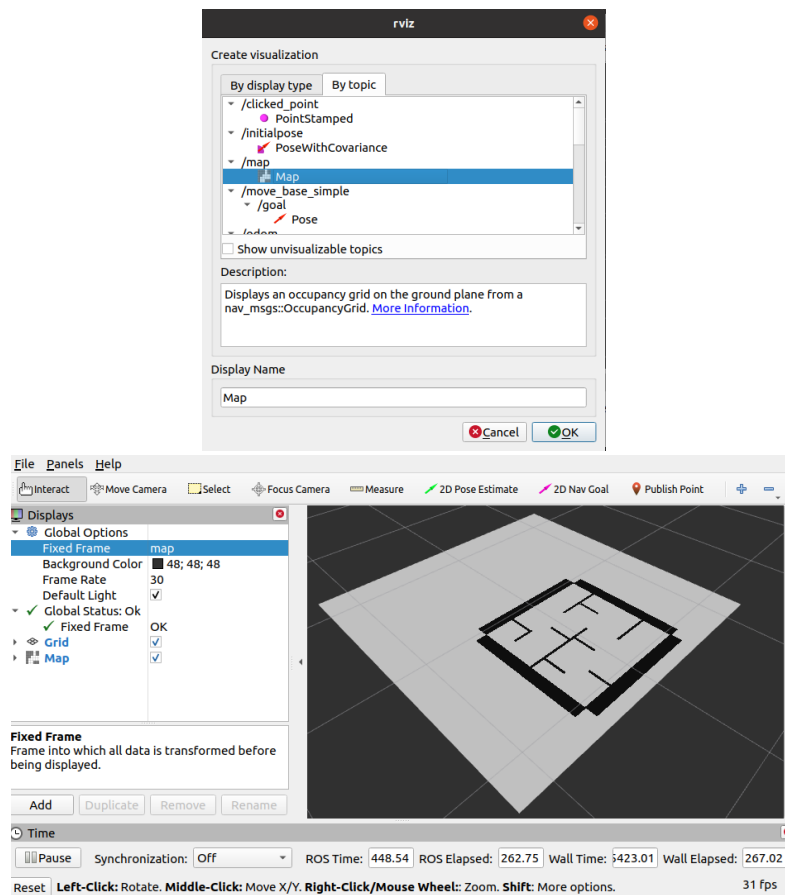
```
>> rosruncatkin_ws/src/anki_description/maps/map.yml
```

و پس از آن در ترمینال دیگر rviz را اجرا می‌کنیم.

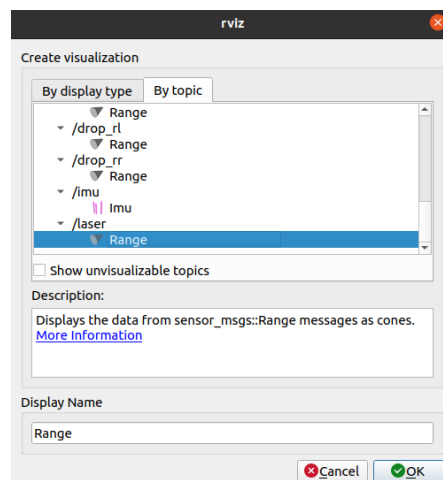


شکل ۳- محیط rviz

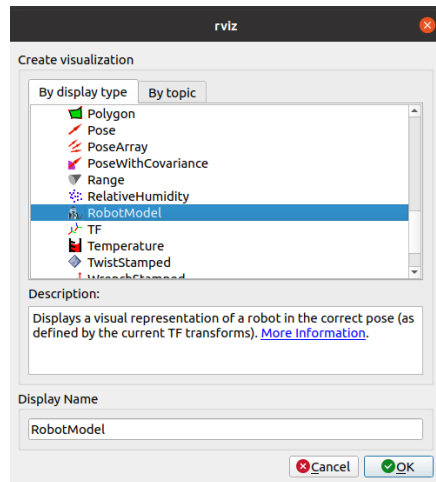
پس از اجرای rviz و همچنین کد مربوط به particle filter لازم است تا topic ها را مطابق دستور زیر اد کنیم.
وارد قسمت add می شویم:



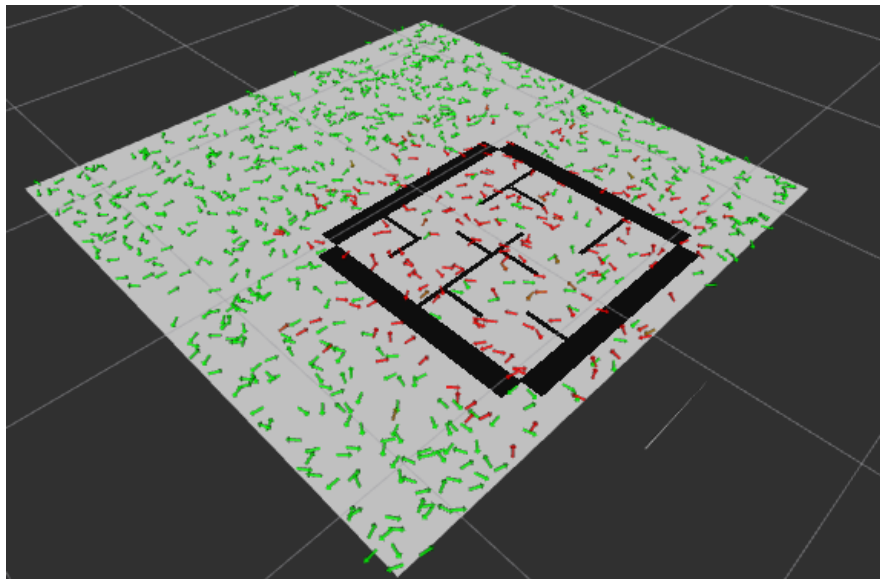
شکل ۴ اضافه کردن نقشه



شکل ۵ اضافه کردن لیزر



شکل ۶ اضافه کردن مدل ربات



شکل ۷ اضافه کردن پارتیکل ها

اکنون به منظور حرکت دادن ربات در محیط از پکیج teleop key استفاده خواهیم کرد تا با حرکت ربات در محیط الگوریتم فیلتر ذرات انجام شود.

```
>> rosrn teleop_twist_keyboard teleop_twist_keyboard.py cmd_vel:=/vector/cmd_vel
```

الگوریتم فیلتر ذرات دارای پیچیدگی‌های زیادی بوده و عملکرد صحیح الگوریتم وابسته به عملکرد صحیح اجزای آن از جمله مدل حرکتی، مدل سنسوری، و غیره می‌باشد. از این رو این الگوریتم به بخش‌های مختلفی تقسیم شده و با پیاده‌سازی هر مرحله، راه برای پیاده‌سازی مراحل بعد باز می‌شود. مراحل مختلف این الگوریتم به صورت زیر شکسته شده‌اند:

۱- شروع ذرات – Particle Initialization

۲- مدل حرکتی – Motion Model

۳- مدل سنسوری ذرات – Particle Sensor Reading

۴- آپدیت وزن‌ها – Weight Update

۵- Resampling

۶- Pose Estimation

۷- بهینه‌سازی پارامترهای نویزی – Noise Implementation

۸- قرارداد بخش‌های مختلف و نهایی کردن الگوریتم

۹- تست الگوریتم

برای هر کدام از بخش‌های مختلف، کدهای تست مختلفی نوشته شده تا اطمینان حاصل شود که هر بخش بصورت جداگانه صحیح عمل می‌کند. در واقع بخش‌های مختلف الگوریتم در غالب همین کدهای تست و با سعی و خطا توسعه پیدا کرده‌اند. این کدها در پوشه test_codes قرار دارند. لازم به ذکر است که هر کدام از این بخش‌ها دارای چالش‌های بسیار زیادی بوده و به مراتب تغییر داده‌شده و بهینه شده‌اند.

در نهایت تمام این کدهای تست کنار هم جمع شده و الگوریتم نهایی را تشکیل می‌دهند.

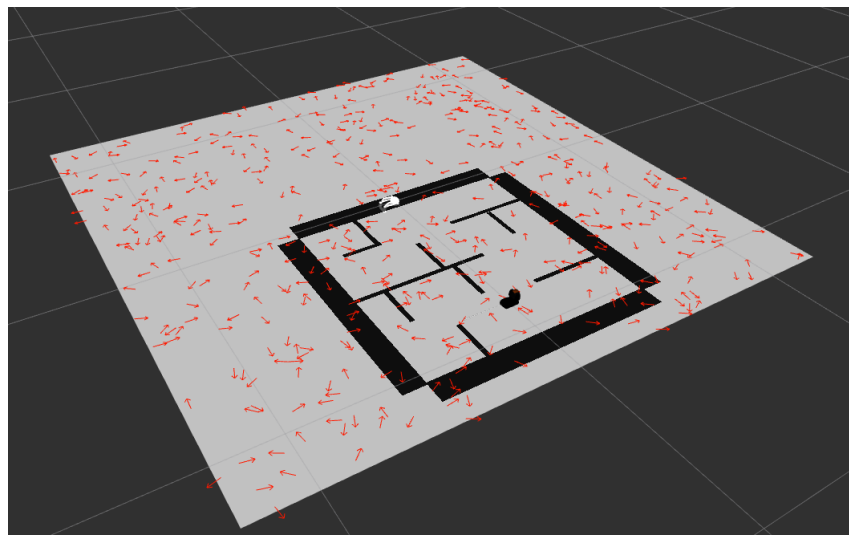
در ادامه هر کدام از بخش‌ها به طور کامل توضیح داده خواهد شد.

شروع ذرات – Particle Initialization

گام اولیه الگوریتم، قرار دادن تعدادی ذره در نقشه تولید شده می‌باشد. برای این منظور با داشتن نقشه و فایل binary_map، کد particle_init نوشته شده تا این ذرات را به طور تصادفی در نقشه قرار دهد. نکته مهم این قسمت این است که ذرات چه در ابتدا و چه در ادامه، فقط در نقاط خالی قرار داده شوند.

در این کد، با بارگذاری نقشه‌ی دودویی محیط، ابتدا سلول‌های آزاد (مقدار ۱) استخراج شده و سپس به صورت تصادفی تعداد ۵۰۰ ذره از بین این نواحی آزاد نمونه‌برداری می‌شود. مختصات پیکسلی هر ذره به مختصات واقعی تبدیل شده و به آن یک زاویه تصادفی بین π^- تا π^+ نسبت داده می‌شود. در نهایت، موقعیت اولیه‌ی ذرات ذخیره شده و به صورت گرافیکی جهت بررسی صحت نمونه‌گیری، روی نقشه ترسیم می‌گردد.

در ادامه لازم است تا این نقاط به همان صورت در محیط rviz نشان داده شوند. برای این منظور کد تست particle_publisher نوشته شده است. این نود ROS با بارگذاری فایل حاوی ذرات اولیه (initial_particles.npy)، آن‌ها را به آرایه‌ای از پیام‌های Pose تبدیل کرده و در قالب یک پیام PoseArray با نرخ ۲ هرتز روی فریم مرجع "map" منتشر می‌نماید. زاویه‌ی هر ذره نیز به کواترنیون تبدیل شده و در قالب پیام geometry_msgs/Pose منتشر می‌گردد. این نود امکان نمایش گرافیکی پراکندگی اولیه‌ی ذرات در RViz را فراهم می‌سازد. نمونه این ذرات در شکل زیر نشان داده شده است:



شکل ۸ - ذرات اولیه در محیط rviz

مدل حرکتی – Motion Model

همانطور که مطلع هستیم، در این الگوریتم نیاز است تا تمام ذرات، حرکتی مشابه حرکت ربات داشته باشند.

برای این منظور کد تست motion_model نوشته شده است. در این کد قصد داریم مدل حرکتی ذرات را طوری طراحی کنیم تا با مشاهده حرکت ربات و حرکت ذرات این اطمینان را حاصل کنیم تا ذرات دقیقاً حرکتی مشابه

ربات دارند. برای مثال اگر ربات به مقدار خاصی جلو می‌رود، تمامی ذرات نیز در راستای خود به همین مقدار جلو بروند. در حین این تست مدل حرکتی ذرات توسعه داده شده که بخش ضروری الگوریتم به‌شمار می‌رود.

همانطور که از قبل اطلاع داریم، مدل حرکتی ربات vector توسط فرمول‌های زیر بیان می‌شوند:

$$x_{t+1} = x_t + \Delta t \times v_t \times \cos(\theta_t + \omega_t \Delta t)$$

$$y_{t+1} = y_t + \Delta t \times v_t \times \sin(\theta_t + \omega_t \Delta t)$$

$$\theta_{t+1} = \theta_t + \omega_t \Delta t$$

پس لازم است تا ذرات نیز از این مدل تبعیت کنند. نکته مهم مدل حرکتی، در نظر گرفتن نویزهای حرکتی می‌باشد که لازم است این نویزها که از توزیع گاوسی پیروی می‌کنند در حرکت اعمال شوند. برای این منظور باید یک واریانس حرکتی برای ذرات در نظر گرفته شود. این واریانس‌ها از تست‌های حضوری فاز دوم پروژه بدست آمده‌اند. در این قسمت؛ این مقادیر صفر در نظر گرفته شده‌اند تا از حرکت صحیح ذرات اطمینان حاصل شود و این مقادیر در بخش بهینه‌سازی پارامترها اعمال خواهند شد.

در کد `motion_model`، دل حرکتی فیلتر ذره‌ای بر اساس مدل سینماتیکی ربات دیفرانسیلی پیاده‌سازی شده است. برای هر ذره، موقعیت فعلی شامل (x, y, θ) ذخیره می‌شود و با دریافت سرعت خطی v و سرعت زاویه‌ای w ، موقعیت جدید ذره پس از مدت زمان Δt محاسبه می‌گردد.

نویز گاوسی به‌صورت مستقل به v و w اعمال می‌شود تا عدم قطعیت حرکتی مدل‌سازی شود. سپس حرکت ذرات به دو حالت تقسیم می‌شود:

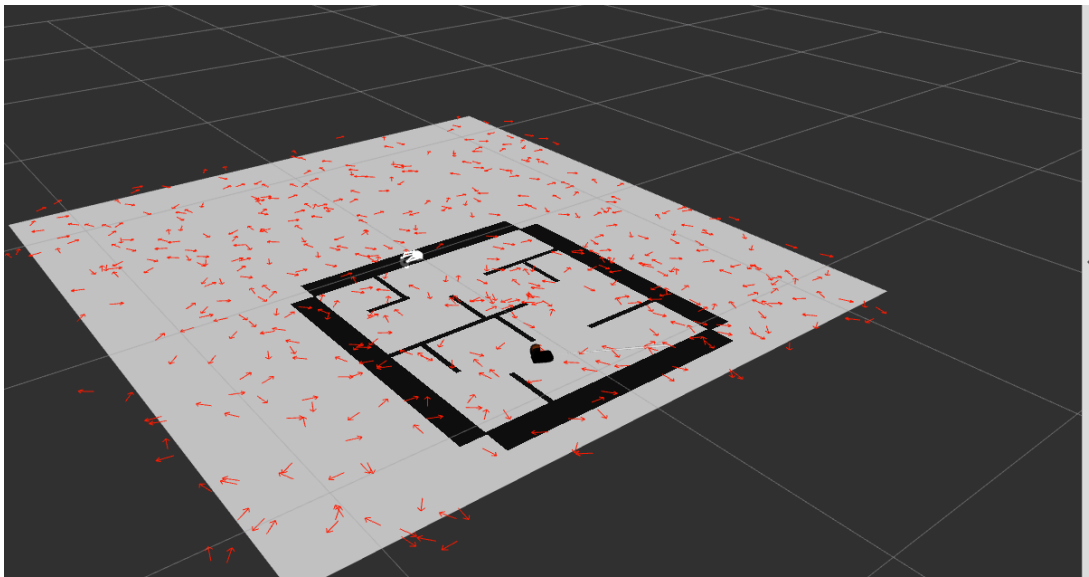
۱- حرکت مستقیم: $w \approx 0$

۲- حرکت منحنی: $w \neq 0$

لازم به ذکر است که برای دادن دستورهای حرکتی به ربات از نود `teleop` استفاده می‌شود.

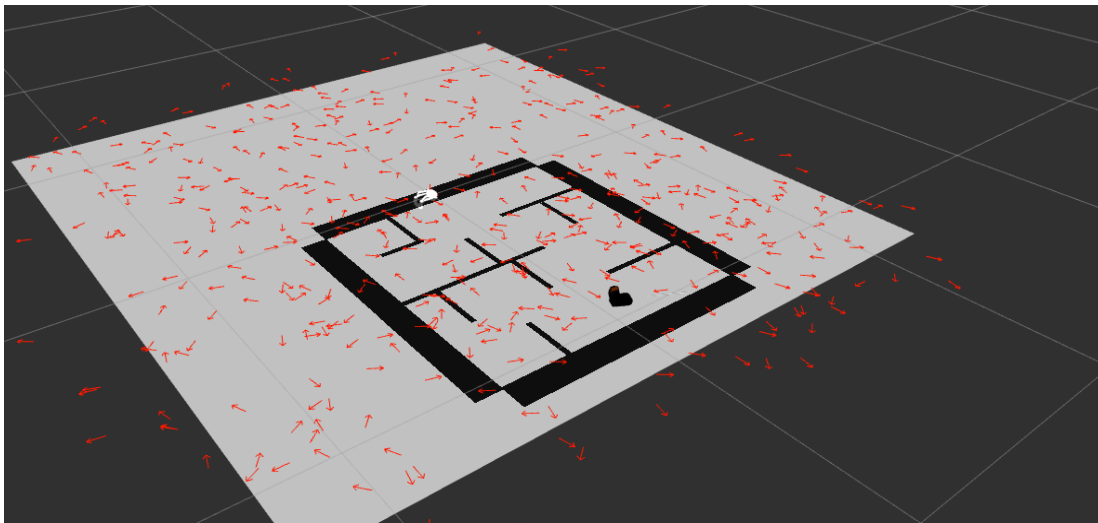
در نهایت هدف این تست این است که آیا حرکت ذرات از حرکت ربات تبعیت می‌کند یا خیر؟

برای این منظور ابتدا حرکت خطی مستقیم رو به جلو را برای ربات در نظر می‌گیریم، در حالت اول ذرات به شکل زیر هستند:



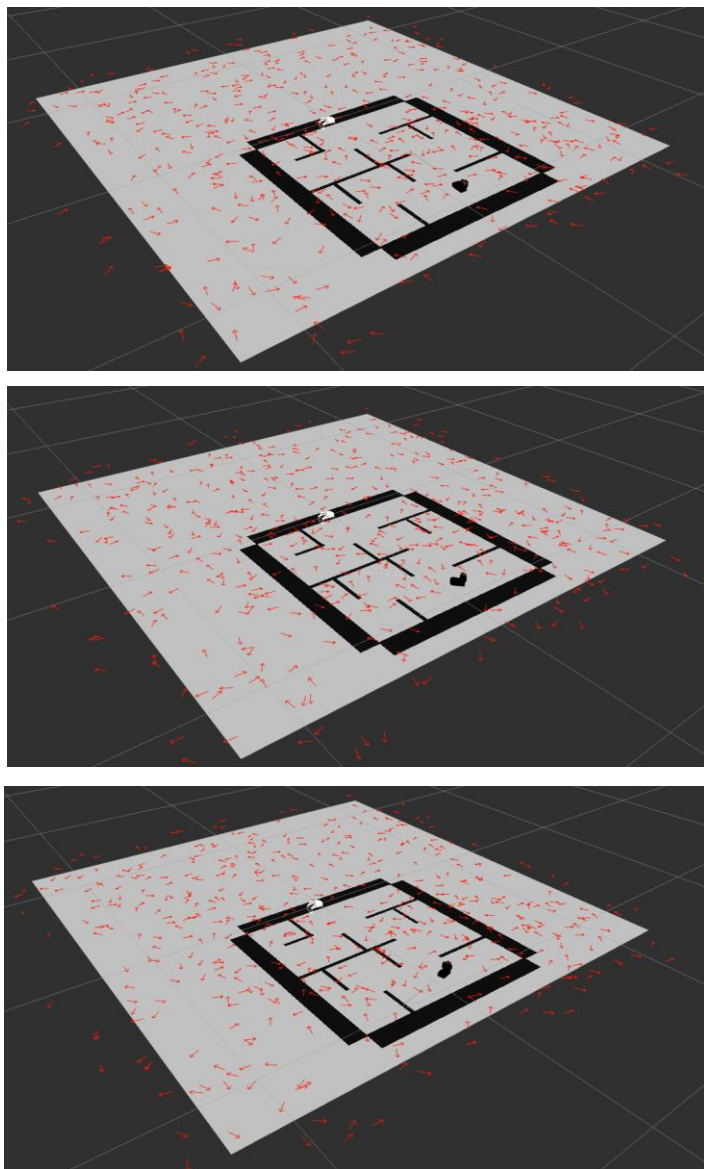
شکل ۹ - بررسی مدل حرکتی ذره‌ها در حرکت مستقیم - حالت اولیه

پس از اعمال کمی حرکت خطی مستقیم به ربات ذرات در حالت ثانویه قرار می‌گیرند



شکل ۱۰ - بررسی مدل حرکتی ذره‌ها در حرکت مستقیم - حالت ثانویه

با دقت و بررسی‌های زیاد در می‌یابیم که اختلاف حرکت ربات در حالت اولیه و ثانویه عیناً در تمام ذرات تکرار شده‌است. در حرکت چرخشی نیز به همین صورت:



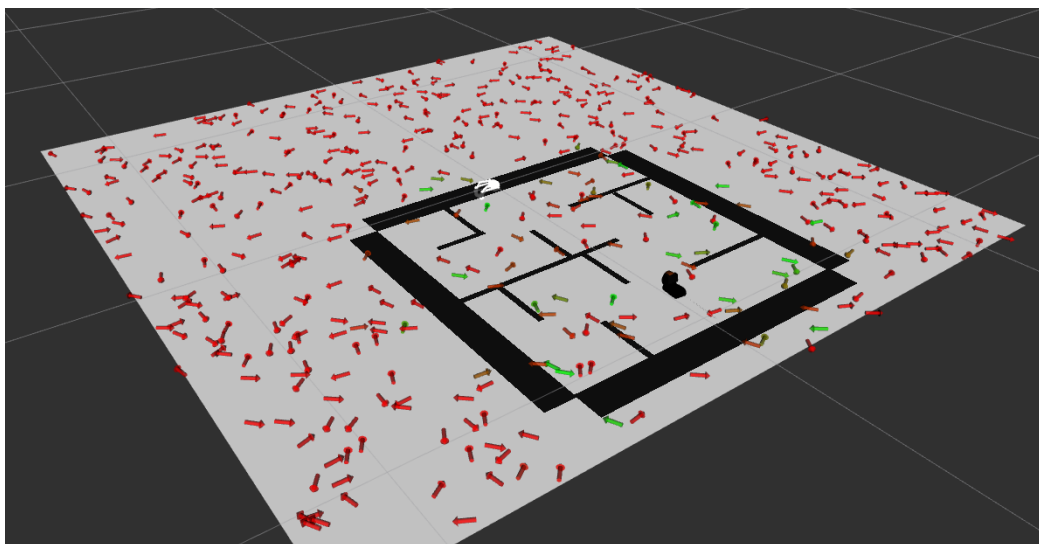
شکل ۱۱ - بررسی مدل حرکتی ذره‌ها در حرکت چرخشی

زاویه چرخش ربات به همان مقدار در ذرات اعمال می‌شود. یکی از مشکلاتی که در روند پیاده‌سازی مدل حرکتی وجود داشت، عدم تبعیت ذرات در حرکت‌های پیوسته و طولانی بود. برای این منظور نرخ به‌روزرسانی ذرات ۲۰ هرتز قرار شده تا ذرات فرصت داشته باشند حرکت خود را با ربات تطبیق دهند.

مدل سنسوری ذرات – Particles Sensor Reading

پیچیده‌ترین و پرمشکل‌ترین بخش این پروژه، پیاده‌سازی مدل سنسوری ذرات می‌باشد. دلیل این پیچیدگی وجود باگ‌ها و مشکلات زیاد، به علت عدم تطابق نقشه در Gazebo و Rviz بود. برای مثال در کدهای اولیه، تمامی ذرات مقادیر ۴۰ سانتی‌متر (حداکثر سنسور فاصله‌سنج ربات) را گزارش می‌دادند، در تحقیقات ثانویه مشخص شد که به علت تفاوت مبدا مختصات در نقشه تولیدشده و محیط rviz، در واقع تمامی ذرات در خارج از محیط دیوارها قرار داشتند (در نقشه مبدا مختصات در پایین و چپ و در rviz مبدا مختصات در بالا و چپ قرار دارد).

برای این منظور، برای تست خوانش فاصله اندازه‌گیری‌شده توسط ذرات، یک ذره دقیقاً در محل ربات قرار داده شده‌است و با ربات حرکت می‌کند (به کمک بخش مدل حرکتی)، انتظار می‌رود که خوانش سنسور این ذره تقریباً در حدود خوانش سنسور ربات باشد. پس از اطمینان از موفقیت این کد ساده، نود `sensor_model_test` نوشته شده‌است. در این کد هدف این است که ذراتی که خوانش سنسور آنها تقریباً برابر ربات است، به رنگ سبز در آیند و بقیه ذرات که خوانش سنسور آنها (مقدار فاصله آنها از مانع) تفاوت زیادی با ربات دارد، به رنگ قرمز باشند. این اولین گام جدی از پیاده‌سازی الگوریتم می‌باشد.

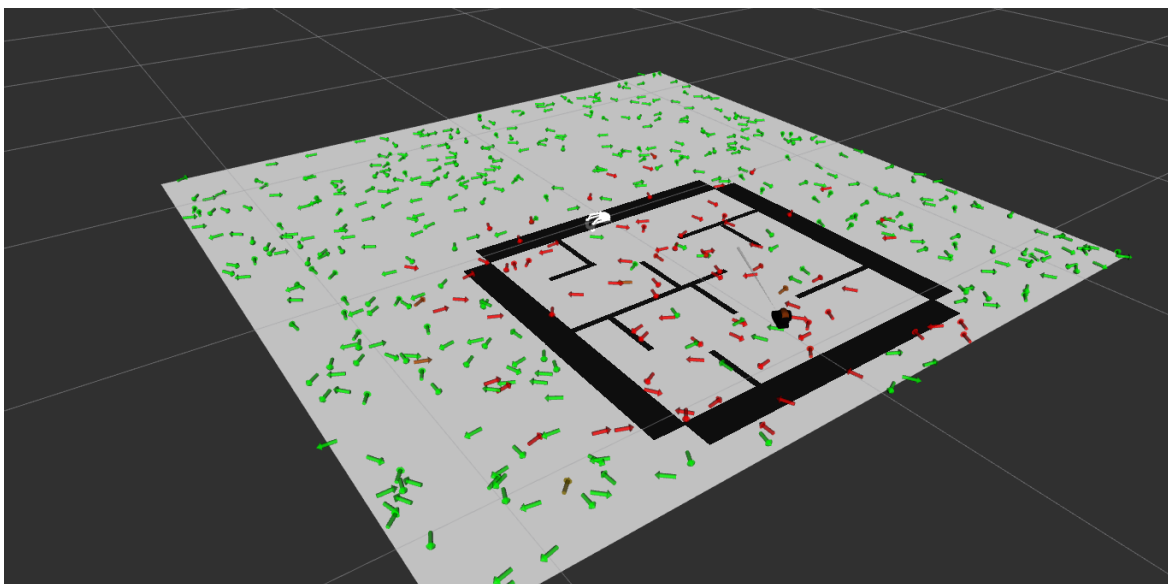


شکل ۱۲ - بررسی مدل سنسوری ذره‌ها - ۱

در تصویر بالا نتایج این تست نشان داده شده‌است. دو نکته مهم لازم به ذکر است:

- ۱- در هر مرتبه، با حرکت ربات، مدل حرکتی باید به ذرات اعمال شود و با خوانش دوباره سنسور رنگ آنها بروزرسانی شود.

۲- با حرکت ذرات ممکن است تعدادی از آنها وارد دیوارها شوند. از آنجا که ضخامت دیوارها نسبت به اندازه ربات و ذرات زیاد است، این امر ممکن است موجب خوانش اشتباه و گمراه کردن الگوریتم شود. نیاز است که به گونه‌ای نوشته شود تا ذراتی که وارد دیوار شده حذف شوند و ذرات جدید جایگزین شوند.



شکل ۱۳ - بررسی مدل سنسوری ذره‌ها - ۲

در تست بالا موقعیت ربات از قصد طوری قرار گرفته تا مانعی را تشخیص ندهد، همانطور که می‌بینیم ذراتی که در بیرون از محیط نقشه قرار گرفته‌اند به علت اینکه مانعی در جلوی خود ندارند به رنگ سبز در می‌آیند که این امر نشان از این دارد که الگوریتم مدل سنسوری ذرات مانند ربات عمل می‌کند. دو نکته اینجا حائز اهمیت می‌باشد:

۱- همانطور که برای مدل حرکتی مقادیر نويز در نظر گرفته شد، مدل سنسوری نیز در ادامه توسط تست‌های فاز دوم پروژه بهینه خواهد شد.

۲- بین مرکز ربات (که مختصات ربات در نظر گرفته می‌شود) و محل سنسور آن حدود 0.043 متر اختلاف می‌باشد که این offset باید در تمامی بخش‌های پروژه در نظر گرفته شود.

بروزرسانی وزن‌ها – Weight Update

در فیلتر ذرات، مرحله‌ی وزن‌دهی (Weight Update) بخش کلیدی و تعیین‌کننده‌ای از کل الگوریتم است. پس از آن که ذرات بر اساس حرکت ربات به‌روز شدند، نیاز است تا مشخص شود کدام ذرات با واقعیت (یعنی داده‌ی

سنسور) هم‌خوانی بیشتری دارند. وزن هر ذره نشان می‌دهد که آن ذره چقدر محتمل است موقعیت واقعی ربات را نمایندگی کند. بنابراین، وزن‌دهی صحیح مستقیماً روی دقت تخمین نهایی و باز نمونه‌گیری مؤثر تأثیر می‌گذارد. برای توسعه روزرسانی وزن‌ها، کد تست weight نوشته شده است.

ابتدا برای هر ذره، خوانش سنسور آن با خوانش سنسور ربات مقایسه می‌شود. اختلاف این دو (یعنی خطای فاصله) وارد مدل گوسی می‌شود و وزن متناسب با آن محاسبه می‌گردد؛ به‌طوری که ذراتی که فاصله‌ی شبیه‌سازی‌شده‌شان نزدیک به فاصله‌ی واقعی باشد، وزن بالاتری می‌گیرند. در صورتی که تمام وزن‌ها صفر شوند (مثلاً به دلیل قرارگیری همه‌ی ذرات در موقعیت‌های نامعتبر یا خطای زیاد)، وزن‌ها به‌صورت یکنواخت بازتنظیم می‌شوند تا فیلتر از هم نپاشد.

در ادامه، این وزن‌ها به صورت رنگی در RViz نمایش داده می‌شوند (سبز برای ذرات با وزن بالا) و چهار مقدار مهم نیز چاپ می‌شود:

- **Max Weight** وزن بیشینه؛ نشان می‌دهد بهترین ذره تا چه حد با سنسور تطابق دارد.
- **Min Weight** بدترین ذره از نظر تطابق.
- **Avg Weight** میانگین وزن‌ها؛ شاخص کلی کیفیت تخمین.
- **Effective Number of Particles (Eff)** این مقدار نشان می‌دهد چند ذره‌ی مؤثر واقعاً در باز نمونه‌گیری سهم دارند. اگر این عدد خیلی کوچک شود، نشان‌دهنده‌ی واگرایی فیلتر است و معمولاً باید باز نمونه‌گیری انجام شود.

```
[INFO] [1752850275.452831, 994.358000]: Applied motion update
[INFO] [1752850275.697668, 994.610000]: Weight stats - Max: 0.0295, Min: 0.0000, Avg: 0.0020, Eff: 49.1
[INFO] [1752850275.746202, 994.653000]: Applied motion update
[INFO] [1752850276.008163, 994.904000]: Weight stats - Max: 0.0300, Min: 0.0000, Avg: 0.0020, Eff: 46.8
[INFO] [1752850276.052621, 994.946000]: Applied motion update
[INFO] [1752850276.304894, 995.212000]: Weight stats - Max: 0.0291, Min: 0.0000, Avg: 0.0020, Eff: 48.7
[INFO] [1752850276.350471, 995.250000]: Applied motion update
[INFO] [1752850276.595325, 995.488000]: Weight stats - Max: 0.0287, Min: 0.0000, Avg: 0.0020, Eff: 50.3
[INFO] [1752850276.651663, 995.539000]: Applied motion update
[INFO] [1752850276.904209, 995.785000]: Weight stats - Max: 0.0296, Min: 0.0000, Avg: 0.0020, Eff: 47.9
[INFO] [1752850276.984702, 995.868000]: Applied motion update
[INFO] [1752850277.234952, 996.111000]: Weight stats - Max: 0.0272, Min: 0.0000, Avg: 0.0020, Eff: 52.3
[INFO] [1752850277.286047, 996.152000]: Applied motion update
[INFO] [1752850277.553645, 996.435000]: Weight stats - Max: 0.0258, Min: 0.0000, Avg: 0.0020, Eff: 55.5
[INFO] [1752850277.598887, 996.467000]: Applied motion update
[INFO] [1752850277.693224, 996.567000]: Applied motion update
[INFO] [1752850277.919327, 996.792000]: Weight stats - Max: 0.0217, Min: 0.0000, Avg: 0.0020, Eff: 60.8
[INFO] [1752850277.970844, 996.827000]: Applied motion update
[INFO] [1752850278.205989, 997.078000]: Weight stats - Max: 0.0239, Min: 0.0000, Avg: 0.0020, Eff: 56.6
[INFO] [1752850278.249966, 997.109000]: Applied motion update
[INFO] [1752850278.478667, 997.341000]: Weight stats - Max: 0.0230, Min: 0.0000, Avg: 0.0020, Eff: 58.5
[INFO] [1752850278.520582, 997.395000]: Applied motion update
[INFO] [1752850278.567592, 997.427000]: Applied motion update
[INFO] [1752850278.829571, 997.701000]: Weight stats - Max: 0.0252, Min: 0.0000, Avg: 0.0020, Eff: 56.1
[INFO] [1752850278.870540, 997.732000]: Applied motion update
```

شکل ۱۴ - آپدیت وزن‌ها

شکل بالا نمونه‌ای از این اعداد را در حین حرکت ربات نشان می‌دهد. از آنجا که فعلاً فرآیند resampling توسعه داده شده است؛ میانگین وزن ذرات روی عددی ثابت قرار می‌گیرد.

بازنمونه‌گیری – Resampling

بازنمونه‌گیری (Resampling) یکی از مراحل کلیدی در فیلتر ذره‌ای است که بعد از مرحله‌ی وزن‌دهی انجام می‌شود. هدف آن همگرا شدن ذرات است، جایی که تنها تعداد کمی از ذرات وزن بالا دارند و بقیه تقریباً نادیده گرفته می‌شوند. در این مرحله، ذرات جدید از مجموعه‌ی فعلی با احتمال متناسب با وزن آن‌ها انتخاب می‌شوند. یعنی ذراتی که وزن بالایی دارند (تطابق بیشتری با داده‌ی سنسور دارند)، احتمال بیشتری دارند که چند بار انتخاب شوند و ذرات با وزن پایین به تدریج حذف می‌شوند.

از نکات بسیار مهم و پارامترهایی که باید با آزمون و خطا تعیین شود، نحوه بازنمونه‌گیری و همین‌طور نرخ آن است. برای مثال با چه مبنایی باید تصمیم گرفته شود که تعداد ذرات کاهش یابد؟ به چه نرخ؟ درباره نحوه پیاده‌سازی resampling در این پروژه در بخش نهایی صحبت خواهد شد.

تخمین موقعیت – Pose Estimation

در فیلتر ذره‌ای، تخمین موقعیت (Pose Estimation) به معنای استخراج موقعیت و جهت‌گیری نهایی ربات از بین مجموعه‌ای از ذرات است. این تخمین، خروجی نهایی الگوریتم است و معمولاً به یکی از روش‌های زیر انجام می‌شود:

۱. میانگین:

موقعیت تخمینی به صورت میانگین وزن دار مختصات تمام ذرات محاسبه می‌شود. یعنی ذراتی که وزن بیشتری دارند، تأثیر بیشتری در نتیجه‌ی نهایی دارند. این روش برای تخمین نرم و پیوسته مناسب است.

۲. ذره با بیشترین وزن:

در این روش، تنها از ذره‌ای که بالاترین وزن را دارد به عنوان موقعیت نهایی استفاده می‌شود. این روش در شرایطی که تخمین دقیق و بدون نویز نیاز است کاربرد دارد، ولی ممکن است نسبت به نویز حساس باشد.

از آنجا که در ربات نرخ نويز زياد بوده، براي تخمين موقعيت ربات از ميانگين ذرات استفاده مي‌شود. به اين صورت که x و y زاويه تخميني بصورت ميانگين تمام x و y زاويه ذرات با وزن بالا در نظر گرفته مي‌شود.

بهينه‌سازي پارامترى نويزي

در ابتدا براي پارامترهايي نظير:

۱- واريانس جابجايي خطي ذرات

۲- واريانس جابجايي چرخشي ذرات

۳- واريانس خوانش‌هاي سنسوري ذرات

از پارامترهاي بدست آمده در فاز سوم پروژه استفاده شد. در ادامه اما مجبور به افزايش اين اعداد به جهت بازيابي بهتر ذرات شديم. در الگوريتم نهايي بسياري از اين پارامترها به صورت Adaptive قرار داده شده‌اند. براي مثال با کاهش تعداد ذرات از ۱۰۰۰ به ۲۰۰ واريانس خوانش سنسوري ذرات نيز کاهش مي‌يابد. اين امر در جلوگيري از پراکندگي ذرات ضروري است.

قراردادن بخش‌هاي مختلف – الگوريتم نهايي

الگوريتم فيلتر ذرات، از تركيب مناسب بخش‌هاي قبلي، به علاوه توسعه بهينه‌سازي‌ها و غيره تشكيل مي‌شود.

لازم به ذکر است که الگوريتم در حدود ۹ نسخه مختلف نوشته شده و نسخه نهايي به عنوان الگوريتم ذرات فيلتر نهايي در نظر گرفته مي‌شود. تفاوت‌هاي اين نسخه‌ها در پيوست آورده شده‌است. کد نهايي به نام `particle_filter` نوشته شده‌است.

در الگوريتم نوشته شده نمونه برداري مجدد به روش سيستماتيک انجام مي‌شود تا از تخریب ذرات، يعني حالي که اکثر ذرات وزن‌هاي نزديک به صفر پيدا مي‌کنند، جلوگيري شود. در اين روش، وزن‌هاي ذرات به صورت تجمعي محاسبه شده و نقاط انتخاب با فاصله‌هاي يکنواخت تعيين مي‌شوند. اين فرآيند ذرات با وزن‌هاي بالاتر را بيشتر حفظ مي‌کند، اما اگر تنها ذرات با وزن بالا نگه داشته شوند، تنوع ذرات کاهش مي‌يابد و الگوريتم ممکن است به يک تخمين نادرست محدود شود. در اين حالت ممکن است ذرات همگي به جايي همگرا شوند که نزديک ربات نباشند. براي حل اين مشکل، فاکتور تنوع (diversity factor) معرفي شده است. اين فاکتور تعيين مي‌کند که چه کسري از ذرات (بين ۰ تا ۱) در هر مرحله نمونه برداري مجدد با نويز تصادفي جابه‌جا شوند تا از تمرکز بيش از حد روي يک منطقه خاص جلوگيري شود. در ابتدای اجرا، فاکتور تنوع ۰.۳ است، يعني تا ۳۰٪ از ذرات با نويز گاوسي (۱۰ سانتي‌متر براي موقعيت و ۱۷ درجه براي زاويه) جابه‌جا مي‌شوند تا فضاي جستجو به‌خوبي کاوش

شود. پس از همگرایی، این فاکتور به ۰.۰۵ کاهش می‌یابد، یعنی تنها ۵٪ از ذرات با نویز کمتر (۲ سانتی‌متر برای موقعیت و ۳ درجه برای زاویه) جابه‌جا می‌شوند تا دقت تخمین افزایش یابد. برای اطمینان از معتبر بودن ذرات جدید، بررسی می‌شود که ذره در فضای آزاد نقشه باینری قرار گیرد، وگرنه جابه‌جایی اعمال نمی‌شود.

خطای موقعیت با فرمول فاصله اقلیدسی ($\sqrt{(x_{est} - x_{true})^2 + (y_{est} - y_{true})^2}$) محاسبه می‌شود، که تخمین وزن‌دار موقعیت ذرات را با موقعیت واقعی اودومتری مقایسه می‌کند. خطای جهت‌گیری نیز با محاسبه حداقل تفاوت زاویه‌ای ($\min(\theta_{err}, 2\pi - \theta_{err})$) به دست می‌آید.

همگرایی زمانی رخ می‌دهد که خطای موقعیت کمتر از ۵ سانتی‌متر و خطای جهت‌گیر کمتر از ۵ درجه به‌مدت ۲ ثانیه پایدار بماند. این معیار نشان می‌دهد که تخمین الگوریتم به موقعیت واقعی ربات بسیار نزدیک شده است. پس از همگرایی، فاکتور تنوع کاهش می‌یابد تا جابه‌جایی‌های تصادفی محدود شوند و ذرات به مناطق با احتمال بالاتر متمرکز شوند. همچنین، تعداد ذرات به‌تدریج از ۱۰۰۰ به ۲۰ کاهش می‌یابد. این فرآیند کاهش نمونه‌برداری (downsampling) هر ثانیه پس از همگرایی انجام می‌شود و شامل انتخاب ذرات با بالاترین وزن‌ها و نرمال‌سازی وزن‌های جدید است. این کار تعداد محاسبات را کاهش می‌دهد و منابع محاسباتی را آزاد می‌کند، در حالی که دقت تخمین حفظ می‌شود.

شرایط توقف الگوریتم زمانی برقرار می‌شود که خطای موقعیت به کمتر از ۱ سانتی‌متر، خطای جهت‌گیری به کمتر از ۱ درجه و تعداد ذرات به ۲۰ برسد. در این نقطه، الگوریتم فرض می‌کند که محلی‌سازی با موفقیت کامل شده است و پیغام زیر چاپ می‌شود:

```
[INFO] [1752855861.446881, 777.905000]: PosErr: 0.012m | AngErr: 4.1deg | Particles: 20 | Div: 0.05 | Eff: 19.3
[INFO] [1752855862.007529, 778.454000]: PosErr: 0.008m | AngErr: 4.4deg | Particles: 20 | Div: 0.05 | Eff: 19.0
[INFO] [1752855862.587867, 779.006000]: PosErr: 0.012m | AngErr: 5.9deg | Particles: 20 | Div: 0.05 | Eff: 14.9
[INFO] [1752855863.140684, 779.554000]:
LOCALIZATION COMPLETE!
[INFO] [1752855863.145506, 779.559000]: Final position error: 0.0055m
[INFO] [1752855863.149521, 779.563000]: Final orientation error: 0.14deg
[INFO] [1752855863.152921, 779.567000]: Final particle count: 20
[INFO] [1752855863.155873, 779.570000]: Localization time: 75.92 seconds
```

شکل ۱۵ – محیط ترمینال پس از موقعیت‌یابی کامل

همچنین ذره‌ای با بالاترین وزن به‌عنوان تخمین نهایی انتخاب شده و به‌صورت بصری با رنگ طلایی در RViz نمایش داده می‌شود تا کاربر بتواند موقعیت نهایی ربات را مشاهده کند.



شکل ۱۶ - ذره با بیشترین وزن به رنگ طلایی

فاکتور تنوع نقش کلیدی در تعادل بین کاوش فضای جستجو و تمرکز بر تخمین دقیق دارد. در مراحل اولیه، مقدار بالای فاکتور تنوع (۰.۳) به الگوریتم اجازه می‌دهد مناطق مختلف نقشه را کاوش کند و از گیر افتادن در یک تخمین نادرست جلوگیری کند. با نزدیک شدن به همگرایی، کاهش این فاکتور به ۰.۰۵ باعث می‌شود ذرات به مناطق محتمل‌تر محدود شوند و دقت افزایش یابد. تنظیم این فاکتور به صورت پویا و وابسته به زمان (با کاهش نمایی در ۳۰ ثانیه) به الگوریتم کمک می‌کند تا به تدریج از حالت کاوش به حالت بهره‌برداری تغییر کند. کاهش تدریجی تعداد ذرات نیز به الگوریتم اجازه می‌دهد تا در ابتدا با تعداد زیاد ذرات (۱۰۰۰) فضای وسیعی را پوشش دهد و پس از همگرایی با تعداد کمتری (۲۰) محاسبات را بهینه کند. مدیریت خطاها، مانند تنظیم زمان گام یا توزیع یکنواخت وزن‌ها، از توقف غیرمنتظره الگوریتم جلوگیری می‌کند و پایداری را در محیط‌های پیچیده تضمین می‌کند.

در نهایت پس از بررسی و تست‌های متنوع می‌توان به نقاط قوت زیر در الگوریتم اشاره کرد:

- امکان کاوش زیاد ذرات در ابتدای نمونه‌برداری باعث می‌شود تا حتی اگر ذره اولیه‌ای منطبق بر ربات نباشد، الگوریتم بتواند موقعیت نهایی را بیابد.

- با کاهش تصاعدی تعداد ذرات از ۱۰۰، سرعت الگوریتم بیشتر می‌شود.

- زیاد بودن نویز سنسور و مدل حرکتی در ابتدای الگوریتم باعث همگرایی سریع‌تر می‌شود.

بسیاری از پارامترهای این الگوریتم با سعی و خطا تنظیم شده‌اند، در زیر لیست مهم‌ترین این پارامترها آورده شده است:

INITIAL_PARTICLES: 1000

توضیح: تعداد اولیه ذرات برای نمایش حالت‌های ممکن ربات.

MIN_PARTICLES: 20

توضیح: حداقل تعداد ذرات که در طول اجرا حفظ می‌شود.

CONVERGED_PARTICLES: 100

توضیح: تعداد هدف ذرات پس از دستیابی به همگرایی اولیه، قبل از کاهش بیشتر.

FINAL_PARTICLES: 20

توضیح: تعداد نهایی ذرات پس از همگرایی کامل و اتمام کاهش نمونه‌برداری.

LINEAR_STD: 0.02

توضیح: انحراف استاندارد نویز گاوسی اضافه‌شده به سرعت خطی در به‌روزرسانی مدل حرکتی (بر حسب متر بر ثانیه).

ANGULAR_STD: 0.05

توضیح: انحراف استاندارد نویز گاوسی اضافه‌شده به سرعت زاویه‌ای در به‌روزرسانی مدل حرکتی (بر حسب رادیان بر ثانیه).

LASER_STD: 0.05

توضیح: انحراف استاندارد مدل گاوسی برای خطاهای اندازه‌گیری حسگر لیزری (بر حسب متر).

MIN_TRANS_THRESH: 0.01

توضیح: آستانه حداقل جابه‌جایی (بر حسب متر) برای فعال کردن به‌روزرسانی حرکتی.

MIN_ROT_THRESH: 0.02

توضیح: آستانه حداقل چرخش (بر حسب رادیان) برای فعال کردن به‌روزرسانی حرکتی.

CONVERGENCE_THRESH: 0.05

توضیح: آستانه همگرایی، که نیاز دارد خطای موقعیت کمتر از ۵ سانتی‌متر و خطای جهت‌گیری کمتر از ۵ درجه باشد (بر حسب متر برای موقعیت).

CONVERGENCE_TIME: 2.0

توضیح: مدت‌زمان (بر حسب ثانیه) که معیارهای همگرایی باید پایدار بمانند تا همگرایی تأیید شود.

STOP_THRESH_POS: 0.01

توضیح: آستانه خطای موقعیت (بر حسب متر، ۱ سانتی‌متر) برای توقف الگوریتم و نشان دادن تکمیل محلی‌سازی.

STOP_THRESH_ANG: 0.0174533

توضیح: آستانه خطای جهت‌گیری (بر حسب رادیان، معادل ۱ درجه) برای توقف الگوریتم.

DIVERSITY_FACTOR_START: 0.3

توضیح: فاکتور تنوع اولیه، که ۳۰٪ از ذرات را در نمونه‌برداری مجدد با نويز جابه‌جا می‌کند تا تنوع حفظ شود.

DIVERSITY_FACTOR_END: 0.05

توضیح: فاکتور تنوع نهایی پس از همگرایی، که میزان ذرات جابه‌جاشده را به ۵٪ کاهش می‌دهد برای محلی‌سازی دقیق‌تر.

نتایج

برای ارزیابی این الگوریتم، ۱۰ تست انجام شده. برای نتیجه این ۱۰ تست سه حالت کلی در نظر گرفته شده است:

۱- موقعیت‌یابی کامل: در صورتی که خطای موقعیت‌یابی کمتر از ۱ سانتی‌متر و ۱ درجه بوده و این فرآیند کمتر از ۹۰ ثانیه طول بکشد.

۲. همگرایی: اگر شرایط موقعیت‌یابی زیر ۹۰ ثانیه انجام نشود، اما خطای الگوریتم بسیار کم باشد.

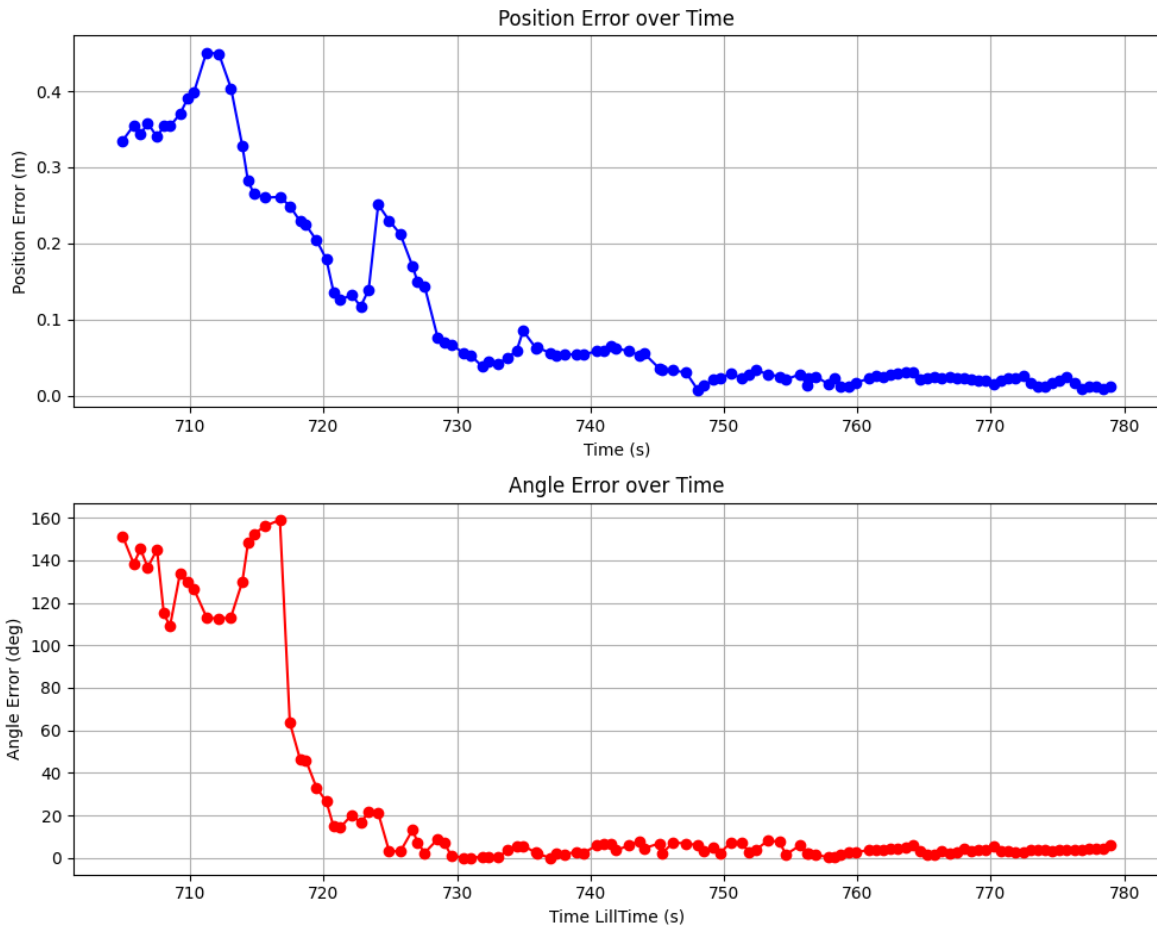
۳. عدم موفقیت: ذرات به نقاط دیگری همگرا شوند و خطا بسیار زیاد باشد.

جدول زیر این ۱۰ تست را توصیف می‌کند:

جدول ۱ - نتایج تست‌ها

تست	حالت نهایی	خطای موقعیت (متر)	خطای زاویه (درجه)	زمان (ثانیه)	تعداد ذره نهایی
۱	موقعیت‌یابی کامل	۰.۰۰۵۵	۰.۱۴	۷۵.۹۲	۲۰
۲	موقعیت‌یابی کامل	۰.۰۰۹۳	۰.۰۷	۶۸.۸۲	۲۰
۳	موقعیت‌یابی کامل	۰.۰۰۶۱	۰.۰۵	۸۰.۸۹	۲۰
۴	عدم موفقیت	۰.۸	۱۵۰	-----	۱۰۰۰
۵	همگرایی	۰.۰۱	۳	۹۰	۲۰
۶	همگرایی	۰.۰۱۶	۶	۹۰	۲۰
۷	عدم موفقیت	۰.۷۱۴	۱۴۵	-----	۱۰۰۰
۸	موقعیت‌یابی کامل	۰.۰۰۹	۰.۲۷	۸۳.۴۲	۲۰
۹	همگرایی	۰.۰۳	۵	۹۰	۲۰
۱۰	موقعیت‌یابی کامل	۰.۰۰۰۹۰	۰.۱۷	۱۳۹	۲۰

فیلم عملکرد الگوریتم که همراه پیوست شده است تست ۱۰ می‌باشد. همچنین نمودارهای خطای آن بصورت زیر می‌باشد:



شکل ۱۷ - نمودارهای خطا در تست ۱۰

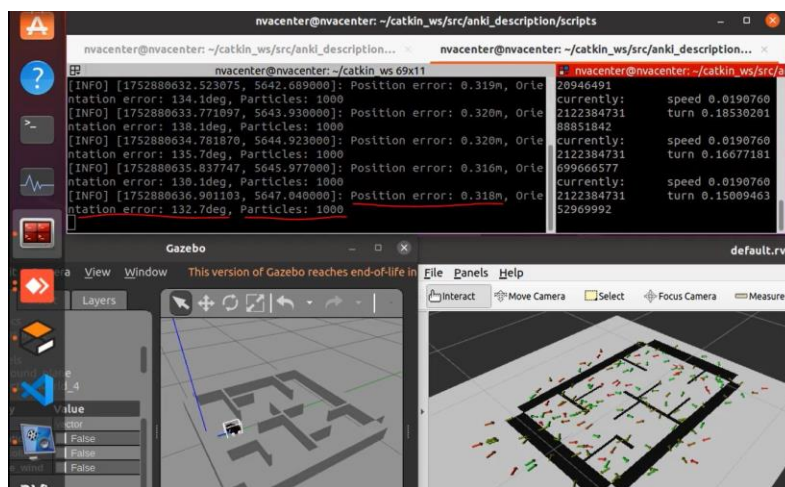
همانطور که مشاهده می‌شود در اکثر مواقع الگوریتم قادر به موقعیت‌یابی ربات می‌باشد. باید توجه داشت که به علت محدودیت‌های سیستم، تعداد ذرات تنها ۱۰۰۰ می‌باشد، با افزایش این تعداد عملکرد الگوریتم بسیار بهتر نیز خواهد شد.

تشخیص kidnapping ربات و بازیابی الگوریتم:

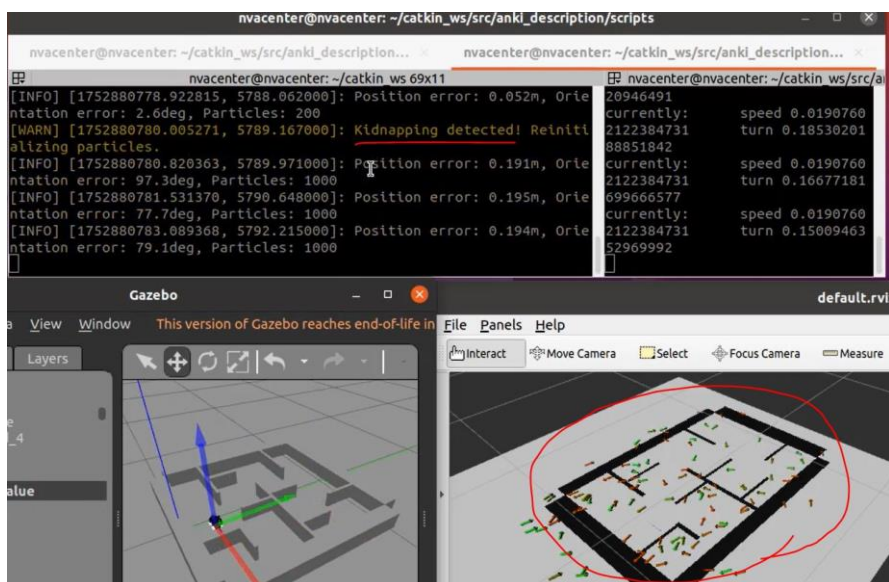
الگوریتم ۱:

در پیاده‌سازی الگوریتم فیلتر ذرات برای تخمین موقعیت ربات، یکی از چالش‌هایی که ممکن است در عمل پیش بیاید، پدیده‌ای به نام kidnapping است. این پدیده زمانی رخ می‌دهد که ربات به طور ناگهانی به موقعیت جدیدی منتقل شود (مثلاً توسط انسان جابه‌جا شود) بدون اینکه سیستم تخمین موقعیت از این جابه‌جایی مطلع باشد. در چنین حالتی، تمام ذرات فیلتر که در اطراف موقعیت قبلی پخش شده‌اند، با داده‌های جدید حسگر ناسازگار خواهند بود و در نتیجه فیلتر دچار خطای شدید تخمین می‌شود. برای حل این مشکل، در کدی که توسعه داده‌ایم با عنوان v20kid.py، بخشی به عنوان تشخیص kidnapping طراحی شده است که به صورت خودکار شرایط مشکوک به جابه‌جایی ناگهانی ربات را تشخیص داده و در صورت لزوم، ذرات را مجدداً مقداردهی اولیه می‌کند.

برای حل این مشکل، در کدی که توسعه داده‌ایم، بخشی به عنوان kidnapping طراحی شده است که به صورت خودکار شرایط مشکوک به جابه‌جایی ناگهانی ربات را تشخیص داده و در صورت لزوم، ذرات را مجدداً مقداردهی اولیه می‌کند. منطق تشخیص در این پیاده‌سازی به صورت زیر است: ابتدا، برنامه بررسی می‌کند که آیا ربات در حالت سکون قرار دارد یا خیر. این کار با استفاده از داده‌های odometry انجام می‌شود. اگر جابه‌جایی ربات از لحاظ موقعیت (dx و dy) و زاویه ($d\theta$) کمتر از آستانه‌هایی مشخص (به ترتیب MIN_TRANS_THRESH و MIN_ROT_THRESH) باشد، فرض می‌شود که ربات متوقف است. این بررسی در هر سیکل اصلی حلقه انجام می‌شود و وضعیت توقف در متغیر robot_stopped ذخیره می‌شود.



شکل ۱۸- در ابتدا ربات شروع به حرکت می‌کند تا موقعیت یابی کند (فاز اول)



شکل ۲۱- پارتیکل ها کانورج میکنند تا دوباره موقعیت یابی از سر گرفته شود(فاز چهارم)

به این ترتیب، فیلتر به جای ادامه دادن با ذرات گمراه، یک نقطه شروع جدید پیدا کرده و با کمک داده‌های حسگر، دوباره موقعیت واقعی ربات را می‌یابد. در این مرحله، همگرایی فیلتر نیز مجدداً از ابتدا آغاز می‌شود و متغیرهای داخلی مربوط به تشخیص همگرایی نیز ریست می‌شوند. برای جلوگیری از اشتباه در تشخیص آدم‌ربایی (مثلاً به دلیل نویز جزئی در داده‌های odometry)، آستانه‌های توقف و آدم‌ربایی به دقت انتخاب شده‌اند. به طور خاص، مقدار $\text{MIN_TRANS_THRESH} = 0.01$ متر و $\text{MIN_ROT_THRESH} = 0.02$ رادیان (حدود 1.15° درجه) باعث می‌شود که حتی جابه‌جایی‌های کوچک به عنوان حرکت ربات تلقی شوند. در مقابل، $\text{KIDNAP_TRANS_THRESH} = 0.05$ متر و $\text{KIDNAP_ROT_THRESH} = 0.2$ رادیان به اندازه‌ای بزرگ هستند که فقط تغییرات ناگهانی و واقعی را تشخیص دهند. در آزمایش‌های اولیه که روی ربات انجام دادم، این روش عملکرد مناسبی داشت. به عنوان مثال، اگر ربات در حالت توقف قرار می‌گرفت و سپس به طور ناگهانی جابه‌جا می‌شد، سیستم بلافاصله این تغییر را شناسایی می‌کرد و بازنشانی ذرات انجام می‌شد. نکته مثبت دیگر این است که چون این تشخیص فقط در حالت سکون بررسی می‌شود، احتمال false positive در حالت حرکت عادی بسیار کم است. در مجموع، این بخش از الگوریتم کمک می‌کند تا تخمین موقعیت در شرایط غیرمنتظره همچنان قابل اطمینان باقی بماند.

ویدیوی این قسمت نیز تحت عنوان kidnapping در پوشه ویدیو ها قرار داده شده است.

الگوریتم ۲:

برای اعمال درست این سناریو در الگوریتم ما لازم است که ابتدا حلقه ی آپدیت کد متوقف شود که با یک کلید k این کار انجام می شود سپس در gazeboo می توان ربات را حرکت داد و با دوباره زدن کلید k آپدیت بدون در نظر گرفتن تغییرات جابه جایی در این مدت از سر گرفته می شود اگر این کار انجام نشود در اودومتری این تغییرات موقعیت هم لحاظ شده و عملاً ربوده شدن با یک تغییر عادی تفاوتی نخواهد داشت.

دو رویکرد کلی برای حل این مسئله می توان در نظر گرفت :

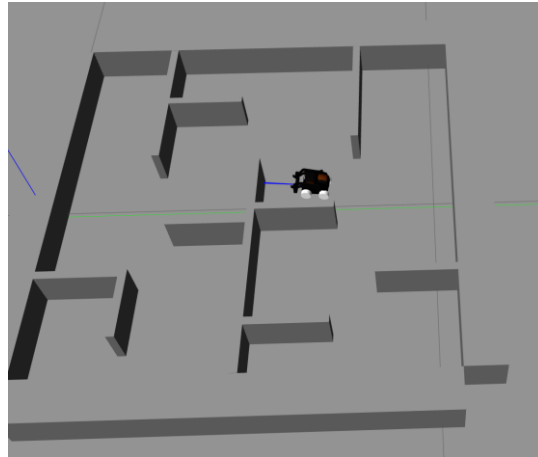
۱. یا باید مکانیزم resampling قوی داشته باشیم به این صورت که در هر مرحله درصدی از ذرات که نتیجه خوبی ندارند حذف شوند و ذرات تصادفی جدیدی اضافه شوند که به مرور با ربوده شدن ربات ذرات موقعیت قبلی کم شده و ذرات جدید اضافه می شوند چون وزن های کمتری به ذرات موقعیت قبلی می رسد همچنین می توان این حذف شدن را به مرور انجام داد به این شکل که برای ذرات فعلی health در نظر گرفت و با هر مرحله نتیجه ی خوب نداشتن از آن کم کرد و با رسیدن به صفر ذرات را به طور کلی باز تعریف کرد و یا با بهبود ذرات به health آن اضافه کرد این رویکرد این مزیت را دارد که ربوده شدن یک حالت خاص نیست بلکه زمانی که خود نمونه گیری هم به خوبی انجام نشده به این شکل شانس اصلاح شدن را خواهد داشت.

۲. تلاش خود را برای شناسایی خود عمل ربوده شدن معطوف می کنیم به این شکل که با توجه به تغییرات نامتعارف در وزن ها (کاهش ناگهانی ، افت زیر یک محدوده خاص) نمونه برداری ذرات را ریست میکنیم و عملاً الگوریتم از اول اجرا می شود.

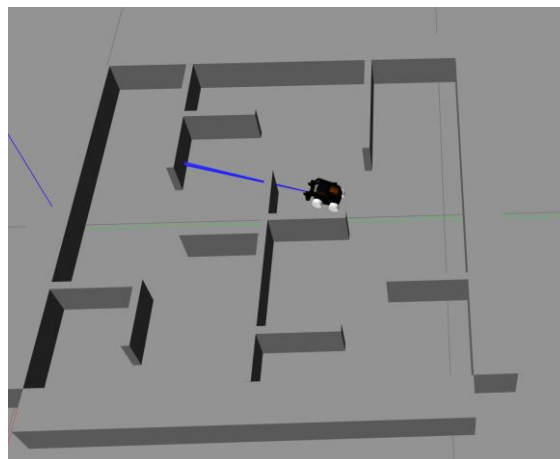
باید توجه داشت که چنانچه هنوز به همگرایی نسبی نرسیده باشیم به شکلی که در موقعیت جدید هم ذراتی باقی مانده باشند نیازی به واکنش اصلاحی نیست و الگوریتم به مسیر معمول خود ادامه می دهد.

یک چالش در این قسمت شباهت زمانی است که سنسور با چرخش از جلوی یک مانع عبور میکند و ناگهان مقدار سنسور به حداکثر خودش میرسد که در این حالت وزن ها به شدت کاهش می یابند که مشابه حالت ربوده شدن می باشد.

همچنین حالتی که سنسور حداکثر مقدار خود را می خواند هم می تواند در صورتیکه موقعیت جدید هم در حالت حداکثر باشد و پیش از آن به یک همگرایی نسبی نرسیده باشیم گیج کننده باشد که البته این حالت جزو هنوز به همگرایی نرسیده ایم لحاظ می شود.



شکل ۲۲ حالت چالشی برای تفکیک از ربنده شدن



انتخاب معیار مناسب برای شناسایی ربنده شدن :

حداکثر وزن ذرات : مناسب نیست زیرا تغییرات کمی از خود نشان می دهد.

میانگین وزن ذرات : قابل تفکیک از حالت چرخش از روبروی یک مانع نیست

میانگین ۲۰ درصد برتر ذرات : عملکرد بهتری نسبت به هر کدام از دو مورد قبلی نشان می دهد زیرا حتی اگر مقدار کمی از ذرات هم همگرا شده باشند و هنوز نقاط پراکنده داشته باشیم همین که درصد بیشتری باشند خود را نشان می دهند.

همچنین یک المان متمایز کننده دیگر نسبت به تغییرات حاصل از چرخش عادی مستمر بودن و زمان بیشتری که برای ریکاور کردن ذرات در حالت ربوده شدن لازم است می باشد.

پس با ترکیب دو تکنیک بالا حالت ربوده شدن را شناسایی و الگوریتم را ریست میکنیم.

جدول ۲ داده های تغییرات میانگین وزن ۲۰ درصد برتر در یک دوران معمولی

زمان	میانگین ۲۰ درصد برتر
۱	۰.۹۷۷۹۹۶۴۶۳۰۲۶۴۴۱۶
۲	۰.۹۶۶۴۰۶۴۲۹۵۷۱۷۰۳
۳	۰.۹۸۷۰۹۹۷۱۱۶۰۴۳۱۶۱۹
۴	۰.۳۱۲۰۳۰۱۴۹۰۴۵۶۶۱۸۴
۵	۰.۸۰۸۷۵۸۳۷۷۹۶۲۱۵۹۹

همانطور که در این داده ها مشخص است تغییرات در اثر حرکت دورانی تنها در نمونه های کمی آن هم در نمونه های با امتیاز ضعیف تر بیشتر خود را نشان می دهند و باعث فعال شدن مکانیزم ربایش نمی شوند.

```
0.9944400965420683
0.9941915881002421
0.9933150828343384
paused for kidanapping!
paused for kidanapping!
```

شکل ۲۳ مکانیزم توقف بروزرسانی

افت در مقدار وزن ها بلافاصله اتفاق نمی افتد بلکه با حرکت در محیط به مرور خود را نشان می دهد.

```
0.9914012549260182
0.9917622129058838
0.992408878802239
0.9826234868309015
0.07312085496363044
[INFO] [1752855785.974597, 418.855000]: KIDNAPPING DETECTED! Mean of top 20%% weights is too low.
0.30601303521790496
0.2643071370852331
[INFO] [1752855786.508603, 419.388000]: Position error: 0.764m, Orientation error: 36.9deg, Particles: 1000
0.31330203124186545
0.23231496536462948
0.5952280759227394
0.6683255870098433
[INFO] [1752855787.683510, 420.562000]: Position error: 0.582m, Orientation error: 112.4deg, Particles: 1000
0.5959292941126
0.6392412348703906
0.5421743301666628
```

شکل ۲۴- داده های تغییرات میانگین وزن ۲۰ درصد برتر در حالت ربوده شدن

زمان	میانگین ۲۰ درصد برتر
۱	۰.۹۹۲۴۰۸۸۸۷۸۸۰
۲	۰.۹۸۲۶۲۳۴۸۶۸۳۰
۳	۰.۰۷۳۱۲۰۸۵۴۹۶۳
۴	۰.۳۰۶۰۱۳۰۳۵۲۱۷
۵	۰.۲۶۴۳۰۷۱۳۷۰۸۵
۶	۰.۳۱۳۳۰۲۰۳۱۲۴۱
۷	۰.۲۳۲۳۱۴۹۶۵۳۶۴
۸	۰.۵۹۵۲۲۸۰۷۵۹۲۲
۹	۰.۶۶۸۳۲۵۵۸۷۰۰۹

در این حالت تغییر شدید تر بوده و تغییرات آن هم بلند مدت تر می باشد.

بنابراین برای تشخیص ربایش یک حد بالای ۰.۶ و یک حد پایین ۰.۰۹ در نظر گرفته شده که چنانچه از ۱۰ نمونه اخیر حداقل ۵ نمونه زیر حد بالا و ۱ نمونه زیر حد پایین باشند برای حالت ۱۰۰۰ ذره ربایش تشخیص داده میشود.

این حد ها اگر زیاد تر از حد کوچک باشند هیچ ربایشی تشخیص داده نمی شود و اگر زیاد تر از حد بزرگ باشند false positive های ما زیاد می شود. بنابراین در تنظیم آن ها باید دقت شود.

در الگوریتم ما resample وجود دارد که تا حدی به ریکاوری پس ربنده شدن کمک می کند.

برای پیاده سازی کد هم از یک deque استفاده شده که در هر مرحله وزن های ذرات مرتب شده ۲۰ درصد برتر انتخاب می شوند و میانگین آن ها به دست می آید سپس این مقدار به deque اضافه شده و با حد ها مقایسه می شود در صورتیکه معیار های ما برآورده شوند درخواست ریست به حلقه ی اصلی برگردانده میشود که مشابه شروع اولیه ی الگوریتم ذرات به صورت یک دست پراکنده تعریف می شوند.

با توجه به ریست شدن الگوریتم عملکرد نهایی مشابه قسمت قبل می باشد.

پیوست – ورژن‌های مختلف الگوریتم

نسخه‌های مختلف به همراه تغییرات اعمال شده

V1: Baseline algorithm

V2: Add downsampling from 1000 to 200, adding logs

V3: Adaptive downsampling, creating criteria to finish the localization

V4: Add Diversity Factor to maintain exploitation, Test diversity based on Resampling

V5: Downsample to 20, Fix Zero Division Error

V6: Increase Position Noise, Test aggressive downsampling

V7: Increase Slope from 20 to 10, Larger perturbation, add recovery mechanism

V8: Fine-tune parameters, increase downsampling rate

V9: Fine-tune parameters

V20kid: Kidnapping detection